

NestJS Practical Assessment

Objective

Build a small school-management backend using NestJS, TypeScript, and PostgreSQL. These technologies are mandatory.

You may choose your ORM or database access library, project structure, and other implementation tools, provided the solution meets the requirements below.

The application manages teachers, classes, and students. Each class has one teacher and a limited capacity. Each student belongs to one class.

Backend only. Keep the solution clean, maintainable, and appropriate to the size of the task.

Core requirements

- Implement REST APIs to create, list, view, update, and delete teachers, classes, and students.
- Store names and emails for teachers and students, and a name and capacity for each class. Include generated identifiers and timestamps.
- Maintain valid relationships and prevent duplicate records.
- Enforce class capacity when adding or moving students and when changing capacity.
- Prevent deletion of records that still have dependent records.
- Return useful validation errors and appropriate HTTP status codes.

Access control

Provide authentication and enforce the following permissions on the server:

- Admin: full access.
- Operator: read, create, and update access; no deletion.
- Student and Teacher: read-only access.

Student and Teacher may read all domain records for this assessment. CSV import is restricted to Admin. Provide demo accounts so each role can be tested.

Choose a suitable authentication approach and explain your decisions. Keep account credentials secure.

Provided data

You will receive `teachers.csv`, `classes.csv`, and `students.csv` containing raw fictional data.

Relationships are expressed using teacher names/emails and class names rather than database IDs. The files are not guaranteed to be clean or internally consistent.

Implement a documented way to import these files into your database. Handle data cleaning, duplicates, conflicts, invalid records, relationship resolution, and class capacity. Preserve the original files. Do not manually repair them or hardcode their contents.

The import must produce a JSON report that accounts for every source row, including its outcome and any reason for skipping or rejecting it. Running the same import again must not create duplicate records.

Decide which corrections are safe and which records should be rejected. Document your policies and assumptions so the results are reproducible. Do not silently discard records or invent missing information.

Quality and documentation

- Separate request handling, business logic, and persistence responsibilities.

- Validate inputs and enforce data integrity.
- Keep secrets outside source code.
- Provide reproducible database setup and meaningful automated tests.
- Expose Swagger UI at `/docs` and an OpenAPI JSON schema at `/openapi.json`. The documentation must support testing authenticated endpoints and reflect actual behavior.

Submission

Submit a Git repository containing:

- Source code and database setup/migrations.
- Tests and `.env.example`.
- A README covering setup, running the application, demo accounts, import execution, testing, and design decisions.
- JSON reports from an initial import into an empty database and a second import into the same database.

Target effort: 8–10 hours. Submit within 72 hours. State your actual time spent, assumptions, and any incomplete requirements. Avoid adding unrelated features.

Review

We will assess correctness, data handling, access control, architecture, tests, documentation, and reproducibility. You will explain your implementation and make a small change during a live review.

Working rules

Write the implementation yourself. Documentation, technical references, Nest CLI scaffolding, and chatbot discussions for explanations or design questions are allowed. Delegating implementation to an AI coding agent or using AI-generated implementation code is not allowed.